Requirements Engineering Specification

Enterprise-Grade Online Banking System (OBS)

color primaryblue

Systems Analysis and Design (SAD) Course Project

Phase 1: Requirements Extraction, Analysis, and Specifications

Student Name:	________________
Student ID:	________________
Instructor:	________________
Academic Term:	Spring Semester, Khordad 1405 (June 2026)

color bordergray Department of Computer Engineering & Information Technology

Contents

1. Introduction

1.1. System Overview

The Online Banking System (OBS) is an enterprise-grade digital banking platform designed to deliver secure, seamless, and high-performance financial services to consumers and corporate entities. OBS operates as the primary digital interface for the bank's services, bridging the gap between legacy core-banking database infrastructures and end-users. The platform provides unified interfaces for personal finance management, large-scale commercial transfers, electronic bill payments, automated notifications, and self-service account administration.

1.2. System Scope & Core Operational Boundaries

The operational scope of the OBS includes the following key application modules:

- **Secure Authentication & Security Auditing:** Multi-layered identity validation, Multi-Factor Authentication (MFA), biometric registration, and immutable security event logs.
- **Account Management & Inquiries:** Real-time account balances, interest tracking, account profile management, and ledger updates.
- **Fund Transfer System:** Immediate and scheduled transfers including intra-bank transactions, ACH (Automated Clearing House) transfers, and high-value wire transfers (RTGS).
- **Utility Billing & Automated Payments:** Integration with third-party billing services (electricity, water, telecommunications) and support for scheduled autopays.
- **Financial Reporting & PDF Statement Engine:** Secure on-demand generation and downloading of legally recognized transaction history reports.

1.3. Target User Base & System Actors

To establish clear requirements, the OBS divides interactions among five primary and secondary actors:

1. **Individual Customer (Primary Actor):** A retail customer seeking personal accounts management, bill payments, small-scale transfers, and statement retrieval.
2. **Corporate Customer (Primary Actor):** A commercial customer executing high-volume ACH/RTGS transfers, requiring advanced dual-authorization workflows and detailed corporate reporting.
3. **System Administrator (Secondary Actor):** An internal technical operative responsible for configuring authorization matrices, system backup protocols, third-party gateway endpoints, and security audits.

4. **Bank Teller/Operator (Secondary Actor):** An internal operational employee with elevated permissions to resolve customer disputes, audit logs, manually override failed transfers, and manage bank branches.

5. **Third-Party APIs (External Actors):** Shetab/RTGS/ACH clearing house APIs, external SMS Gateway microservices, and identity validation databases.

2. Functional Requirements

Functional requirements represent the core actions, calculations, and services the system must execute. They are classified into eight distinct operational categories. All system requirements are written using strict engineering "**shall**" terminology to define absolute system obligations.

To ensure pristine layout presentation and prevent arbitrary page breaks, the requirements matrix is presented in two balanced matrices using professional, borderless table styles.

Table 1: Online Banking System - Functional Requirements Specification (Part 1)

Req. ID	Category	User Requirement	System Requirement ("Shall" wording)
OBS-FR-01	Authentication	Users must be able to register secure accounts online.	The system **shall** provide a customer registration interface that validates the email, phone number, and national ID, and cryptographically hashes the password prior to database persistence.
OBS-FR-02	Authentication	Users must log in securely with two-factor validation.	The system **shall** enforce secure multi-factor authentication (MFA) by requiring a 6-digit Time-Based One-Time Password (TOTP) or SMS token after primary credential validation.
OBS-FR-03	Authorization & Access Control	Different user roles must have distinct and restricted platform privileges.	The system **shall** enforce Role-Based Access Control (RBAC) schemas, dynamically rendering the dashboard and features according to the authenticated user's role (Customer, Teller, or Administrator).
OBS-FR-04	Authorization & Access Control	Administrative logins must be audited, and idle sessions terminated.	The system **shall** automatically invalidate and terminate any administrative user session that remains inactive for more than 10 consecutive minutes.
OBS-FR-05	Data Processing	Customers should see updated account ledger data immediately.	The system **shall** execute ACID-compliant database transaction queries, instantly updating checking and savings ledgers upon completion of a debit/credit operation.
OBS-FR-06	Data Processing	User input must be checked for safety before processing.	The system **shall** sanitize and validate all client-side form fields and query strings on the server side to protect against SQL Injection, XSS, and buffer overflow attempts.
OBS-FR-07	UI/UX	The application must work on mobile, tablet, and desktop screens.	The system **shall** utilize fluid grid layouts and responsive CSS media queries, rendering dashboards consistently across viewport resolutions from 320px to 2560px.
OBS-FR-08	UI/UX	Visually impaired users must be able to read screens easily.	The system **shall** conform to the W3C Web Content Accessibility Guidelines (WCAG) 2.1 AA standards, supporting high-contrast styling and screen readers.

Table 2: Online Banking System - Functional Requirements Specification (Part 2)

Req. ID	**Category**	**User Requirement**	**System Requirement ("Shall" wording)**
OBS-FR-09	Reporting	Customers should be able to download their statement documents.	The system **shall** compile checking and savings transaction history into a secure PDF format on demand, initiating the browser download immediately.
OBS-FR-10	Reporting	Managers must get automated summaries at month-end.	The system **shall** compile a consolidated bank-wide monthly financial ledger report at 23:59:59 on the last day of each month and dispatch it to the management team.
OBS-FR-11	Third-Party Integrations	Customers must be able to transfer money to external bank accounts.	The system **shall** communicate with the national Shetab and RTGS clearing house API endpoints using RESTful JSON packages secured via private TLS certificates.
OBS-FR-12	Third-Party Integrations	Users must receive transaction confirmation messages immediately.	The system **shall** execute an asynchronous microservice call to the external SMS Gateway API within 2 seconds of any successful transaction exceeding $10.00.
OBS-FR-13	Error Handling & Logging	Users must be clearly informed when operations fail.	The system **shall** display a modal window with a non-technical error description and actionable resolution steps when a network timeout or data validation exception occurs.
OBS-FR-14	Error Handling & Logging	Tech teams must have detailed security logs for errors.	The system **shall** write all security-sensitive events, database transaction failures, and validation errors into a centralized log management server using structured JSON formatting.
OBS-FR-15	Backup & Restore	Financial ledgers must be backed up daily to prevent data loss.	The system **shall** run an automated daily snapshot backup of the relational database at 02:00:00 local time, writing encrypted backups to secure offsite cloud storage.
OBS-FR-16	Backup & Restore	A recovery site must be maintained for rapid restoration.	The system **shall** support warm-standby database replication to a geographically separate data center, allowing recovery within 30 minutes in case of disaster.

3. Non-Functional Requirements

Non-functional requirements specify quality attributes, constraints, performance benchmarks, and compliance limits. To ensure high-quality standards, all NFRs are expressed as highly quantitative and testable engineering specifications.

Table 3: Online Banking System - Non-Functional Requirements Matrix

Req. ID	**Category**	**User Requirement**	**System Requirement (Quantitative & Testable)**
OBS-NFR-01	Usability	The user dashboard must be highly intuitive.	The application dashboard **shall** be designed such that first-time users can complete their initial transfer setup in less than 3 minutes, with a task success rate greater than 95%.
OBS-NFR-02	Security	Sensitive customer and bank database files must be encrypted.	The system **shall** encrypt all persistent database storage volumes at rest using the AES-256 standard and secure all APIs and communication channels in transit using TLS 1.3.
OBS-NFR-03	Security	Passwords must be protected using modern hashing.	The system **shall** hash all user-entered passwords prior to database storage using the Argon2id cryptographic hashing algorithm with a minimum memory cost of 64MB.
OBS-NFR-04	Reliability	Transactions should never result in partial states or lost data.	The system database engines **shall** guarantee full transactional integrity, enforcing ACID principles to automatically roll back any transaction that fails mid-execution.
OBS-NFR-05	Performance	Page loads must be fast even under heavy usage.	The system **shall** compile and render individual user dashboard pages in less than 2.0 seconds under normal conditions and less than 3.0 seconds under peak load.
OBS-NFR-06	Scalability	The system should support many users concurrently.	The platform application architecture **shall** maintain standard operation at up to 100,000 concurrent active user sessions without showing throughput degradation.
OBS-NFR-07	Availability	The online banking portal must stay online constantly.	The system **shall** maintain an overall service availability uptime of 99.99% over any given calendar month, restricting planned downtime to pre-announced hours.
OBS-NFR-08	Compatibility	The portal must work identically on all devices.	The application **shall** render and perform identically on Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge, Android (v9.0+), and iOS (v14+).

4. Form-Based Specifications for Core Transactions

To ensure detailed technical design clarity, the top five core system transactions are fully mapped into Structured Natural Language form-based tables using professional horizontal-bordered tables.

4.1. Core Transaction 1: User Registration & MFA Setup

Structured Form-Based Specification: OBS-SRS-FR-01.00	
Requirement ID	OBS-SRS-FR-01.00
Function / Entity	User Registration & Multi-Factor Authentication (MFA) Initialization
Description	Registers a new retail customer on the OBS platform and sets up their SMS-based or TOTP-based Multi-Factor Authentication.
Inputs	National ID number, primary mobile number, email address, password string, and OTP confirmation.
Source	Customer UI Registration Form and SMS/Authenticator Client.
Outputs	Newly created Customer Profile record, active database credential row, and screen validation banner.
Destination	Core Banking Database and User UI Screen.
Action to be Taken	1. Verify that the National ID is unique and not already registered. 2. Check password strength using regex: must contain at least 8 characters, a number, a special character, and an uppercase letter. 3. Dispatch a 6-digit verification code to the registered mobile number. 4. Validate user confirmation input; block process if incorrect code is entered thrice. 5. Cryptographically hash the validated password using Argon2id. 6. Initialize account state as 'Pending Verification' and flag MFA as 'Active'.
Pre-conditions	User is not authenticated; registration page is open; system is online.
Post-conditions	Database contains a new, securely hashed, and pending-verification customer record.
Side Effects	Sends confirmation registration SMS to the user via SMS Gateway API.

4.2. Core Transaction 2: Secure Login & Session Initialization

Structured Form-Based Specification: OBS-SRS-FR-02.00	
Requirement ID	OBS-SRS-FR-02.00
Function / Entity	Authentication and JWT Session Token Generation
Description	Authenticates users using passwords and dynamic MFA verification, and initializes a highly secure JWT session.
Inputs	User Identifier (National ID/Username), password string, and dynamic 6-digit MFA OTP code.
Source	Secure Web/Mobile Login Form and SMS/MFA app.
Outputs	Dynamic cryptographic JWT Session Token and dynamic custom dashboard UI rendering.
Destination	Client-side browser cookies (secure, HTTPOnly) and core database session tables.
Action to be Taken	1. Query database to retrieve stored credential data matching the user identity. 2. Compare user password hash against database record using Argon2id check. 3. If primary credentials match, trigger SMS/TOTP challenge. 4. Confirm the dynamic MFA input matches the computed server value. 5. Generate a JWT Session Token containing roles, permissions, and an expiration timestamp (30 mins). 6. Return authorization token and redirect the user to their designated dashboard.
Pre-conditions	User is unauthenticated; system has internet connectivity; user is registered.
Post-conditions	Client holds a valid, secure JWT session token; system records active session metrics.
Side Effects	Logs authentication timestamp and client IP; invalidates any stale user session.

4.3. Core Transaction 3: Domestic Fund Transfer (Intra-Bank & ACH/Wire)

Structured Form-Based Specification: OBS-SRS-FR-03.00	
Requirement ID	OBS-SRS-FR-03.00
Function / Entity	Fund Transfer Core Transaction Handler
Description	Safely debits a customer's checking account and credits a destination bank account (internal or external clearing house) in an ACID-compliant transaction.
Inputs	Source Account Number, Destination IBAN, Transfer Amount, Transfer Type (Intra-bank, ACH, or RTGS), and Transaction PIN.
Source	Transfer Request UI Form and Secure Authentication Core.
Outputs	Updated balances in source/destination accounts, database ledger logs, and dynamic receipts.
Destination	Core Accounts Database, Shetab API Gateway, and User Receipts UI.
Action to be Taken	1. Retrieve source account status; confirm checking account is active and unrestricted. 2. Verify source balance is sufficient for amount + transfer fee. 3. Check destination IBAN format. If internal, verify account exists in active database. 4. Perform database row locking to prevent race conditions during state transfer. 5. Debit source checking account balance and update transaction logs. 6. If internal, credit destination account. 7. If external, route structured REST package to Shetab Gateway. 8. Generate transaction receipt and commit the ledger database modifications.
Pre-conditions	User is securely authenticated; session is active; daily limits are not exceeded.
Post-conditions	Source checking balance is debited; target balance is updated; database states are finalized.
Side Effects	Triggers transaction confirmation SMS alerts to both sending and receiving parties.

4.4. Core Transaction 4: Automatic Bill Payment & Utility Registration

Structured Form-Based Specification: OBS-SRS-FR-04.00	
Requirement ID	OBS-SRS-FR-04.00
Function / Entity	Bill Payment Engine
Description	Links utility billers to customer profiles and processes recurring or on-demand bill pay cycles.
Inputs	Customer Account Number, Biller ID, Bill Payment Token, and Scheduled Date.
Source	Customer Bill Pay Dashboard and External Utility APIs.
Outputs	Direct payments to utility provider registries and digital confirmation receipts.
Destination	Core Banking Database and External Utility Biller Gateways.
Action to be Taken	1. Connect to the designated utility biller database to fetch current outstanding amounts. 2. Validate checking account active state and verify sufficient funds exist. 3. Lock account ledger to process the transaction securely. 4. Debit account ledger by the specified bill amount. 5. Issue SOAP/REST request to utility gateway confirming immediate payment settlement. 6. Store payment logs and render the digital payment confirmation details.
Pre-conditions	Customer session is validated; biller account details are matched.
Post-conditions	Customer balance is reduced by bill amount; utility bill status shows as 'Settled'.
Side Effects	Updates recurring calendar trackers; sends notification receipt via secure email.

4.5. Core Transaction 5: Statement Generation & PDF Export

Structured Form-Based Specification: OBS-SRS-FR-05.00	
Requirement ID	OBS-SRS-FR-05.00
Function / Entity	PDF Document Compilation Engine
Description	Queries transaction tables for specified date ranges, compiles chronological data, formats, and generates a downloadable PDF file.
Inputs	Account Number, Start Date, End Date, and Export Format (PDF/CSV).
Source	Customer Reports UI Page and Database Transaction History Tables.
Outputs	Securely structured, readable PDF report and download streams.
Destination	Client UI Interface and Web Download Pipeline.
Action to be Taken	1. Check user session to confirm ownership rights over the requested account number. 2. Run query to fetch all transaction ledger records matching dates. 3. Parse raw data to calculate aggregate starting balance, total deposits, and withdrawals. 4. Feed parsed data array to the server-side PDF generator engine. 5. Generate standard PDF layout complete with bank logos, watermarks, and verification signatures. 6. Stream the generated binary document back to client browser, initiating automatic download.
Pre-conditions	User is successfully logged in; date parameters are valid (Start Date $\leq$ End Date).
Post-conditions	Database states remain unchanged; client successfully downloads the PDF file.
Side Effects	Logs document creation metrics in the system logs for administrative security auditing.

5. Graphical UML Notations

To align with structural engineering practices, we provide graphical models illustrating system boundaries, actors, and transactional event message flows.

5.1. UML Use Case Diagram

The following diagram models the boundaries of the Online Banking System and maps the operational associations between primary/secondary actors and core use cases.

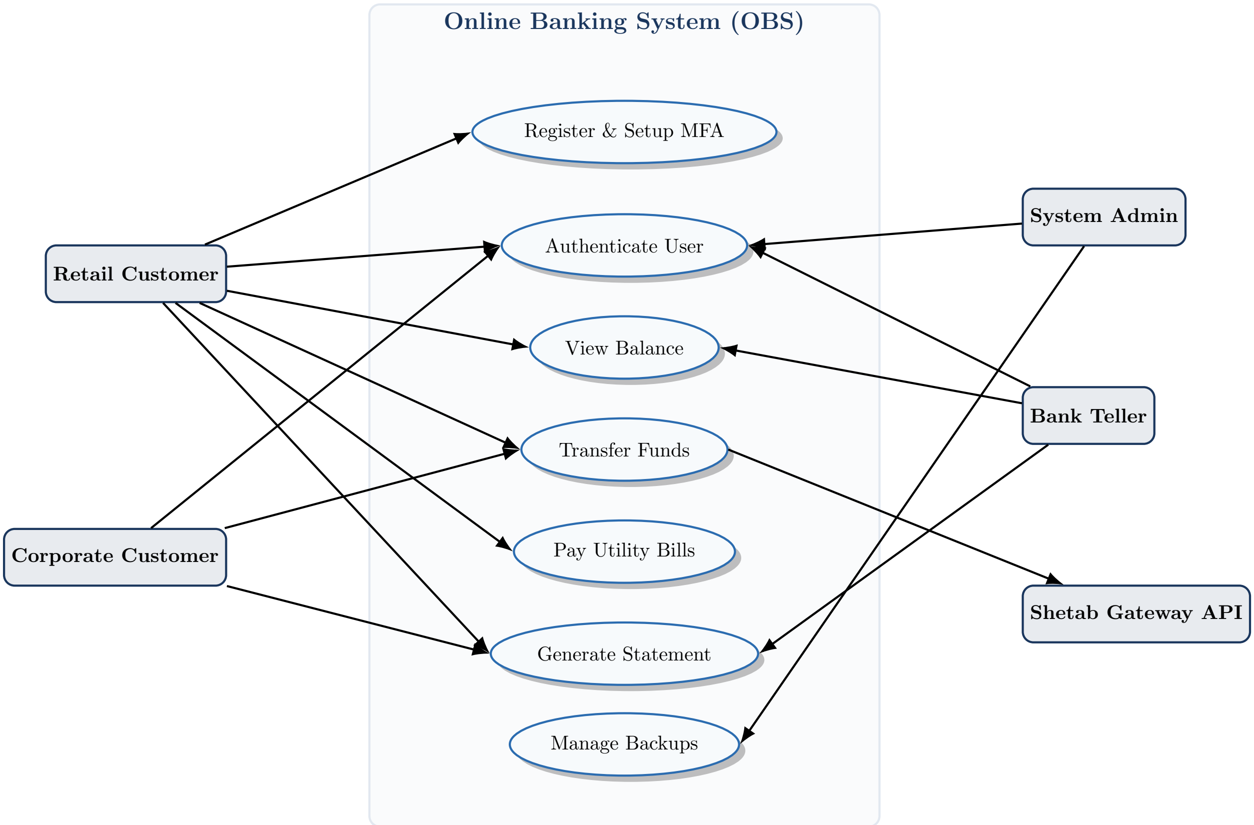

Figure 1: UML Use Case Diagram: User boundaries and actors mapping.

5.2. UML Sequence Diagram (Domestic Fund Transfer Workflow)

The sequence diagram chronologically traces message exchanges across primary components during a fund transfer event.

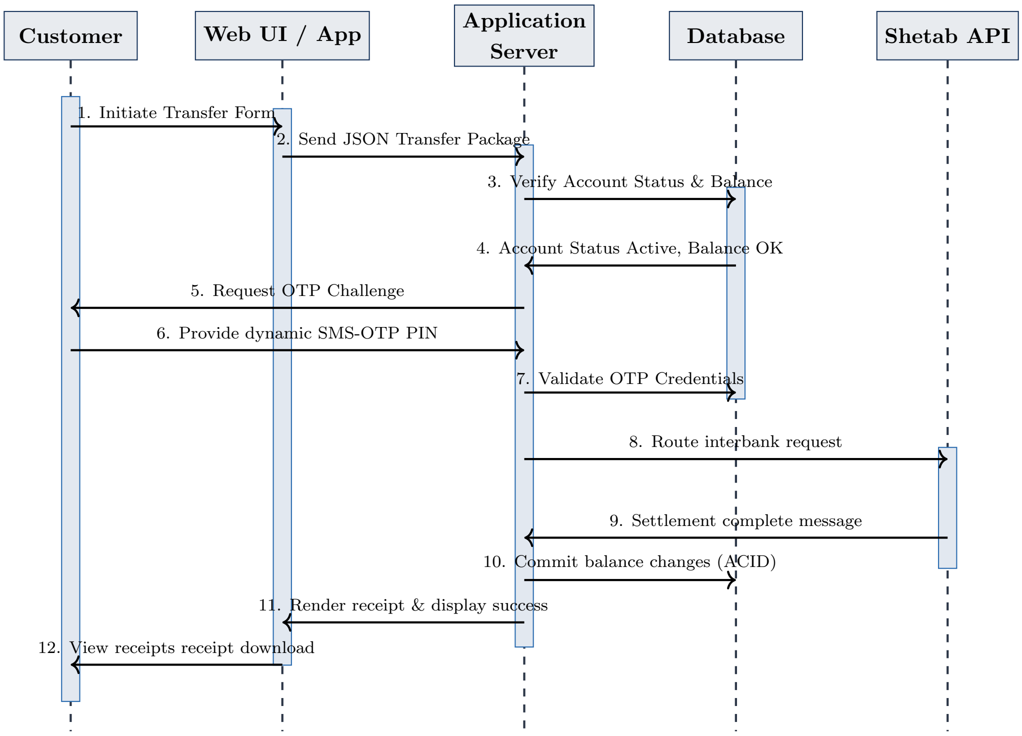

Figure 2: UML Sequence Diagram: Chronological flow of domestic fund transfer transaction.